The Mario Bros

Alex Aoki, Zach Atalay, Nico DiLullo, Theo Romero

BROWN

CSCI 1470

Background

Inspired by the success of the Dreamer series of models, we wanted to train a series of models to play a video game we all had fond memories of: Super Mario Bros. We wanted to create and test a series of models to try and beat the game, with an emphasis on performance measured in two ways: success and efficiency (in terms of training and inference).

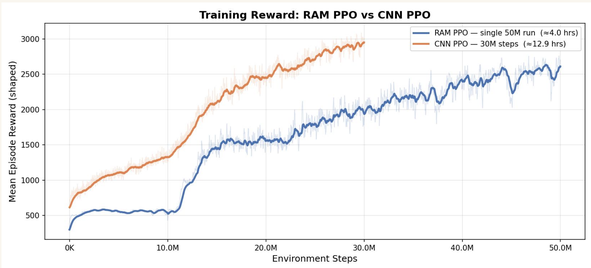
The Problem

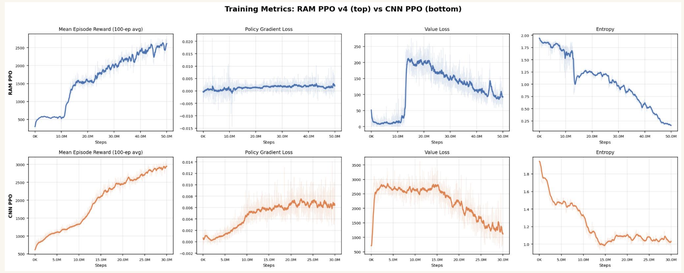
We need to train a Super Mario Bros agent on raw pixels and a reward signal. The agent receives 4×84×84 preprocessed grayscale frames and chooses from 7 discrete actions. The difficulty is that raw game rewards are sparse, the state space is enormous, and the underlying environment uses a deprecated OpenAI Gym API incompatible with modern RL libraries like Stable Baselines 3. This naturally raises questions about custom reward design, optimality, and more.

Takeaways

1. Some "vision" problems can be collapsed to MLPs by abstracting states to compact, low-dimensional state representations (e.g., RAM)
2. This provides enormous efficiency gains, with minimal (if any) performance degradation
3. Custom reward shaping significantly improve performance
4. Remaining questions about what makes a good policy
5. Multi-agent collaboration training was not doable with Gym (for this game)

Training





Training is incredibly slow. Parallelism helps (you can run multiple environments at the same time), but training runs are still take ~4 hours for our RAM PPO and 15+ for our CNN PPO. Notably, training was only feasible with custom reward shaping.

Methodology, Experiments, Etc.

CNN PPOs

Four stacked grayscale frames are processed through a three-layer CNN into a 512-dim shared representation, which feeds separate policy and value heads trained end-to-end with PPO.

DQNs

We found DQNs to be very sample inefficient, and had parallelization issues. Based on this, we estimate it would have taken ~75 hours to train a performant DQN. Tried to mitigate with custom rewards.

RAM PPOs

128-dimensional NES RAM features, encoding Mario's position, velocity, enemy locations, and game state are passed through a two-layer MLP into separate policy and value heads trained with PPO.

Other Optimizations

We also tried quantizing to various precisions, compiling, custom rewards, entropy tuning, and various vectored training optimizations.

Model Specs & Inference

Metric	DQN	RAM PPO	CNN PPO
Inferences/sec	2,285	**36,410**	1,787
Latency (μs)	437.6	**27.5**	559.5
Inference Latency (ms/step)	0.69	**0.04**	0.65
Parameters	1,687,719	**199,432**	2,213,544
Obs. Shape	(4, 84, 84)	(128,)	(4, 84, 84)

Progression

Greedy

Model	Reward	x_pos	Done%
DQN	139 ±0	594	0%
RAM PPO	**3059 ±0**	**3161**	**100%**
CNN PPO	2459 ±0	2914	0%

Stochastic

Model	Reward	x_pos	Done%
DQN	1694 ±594	1128	0%
RAM PPO	2246 ±803	2341	40%
CNN PPO	**3008 ±122**	**3120**	**90%**

It is not immediately clear which policy is preferable, as this depends on the evaluation metric. RAM PPO achieves the strongest greedy performance (best-action quality), while CNN PPO performs better under stochastic evaluation, indicating a more robust and well-calibrated action distribution.

References

[1] Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). OpenAI Gym. arXiv preprint arXiv:1606.01540.
[2] DreamerV1 Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2020). Dream to control: Learning behaviors by latent imagination. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/1912.01603
[3] A full list of references can be found in the bib.tex file in our repository.

Acknowledgements

Thanks to the entire CS1470 staff for an awesome semester, OSCAR and GoogleColab for compute resources, and Nintendo for SuperMario.